YAPAY ZEKÂ İLE PROGRAMLAMA DERS UYGULAMALI ÖDEV

YAPAY ZEKÂ DESTEKLİ SQL ASİSTANI LABORATUVARI

SÜRE
120 dakika

Doğal dilden SQL üreten yapıyı canlı veritabanına bağla, güvenlik filtrelerini kur, otomatik hata düzeltmeyi entegre et ve test et.

Senaryo

Bir yazılım şirketinde Veritabanı ve Yapay Zekâ Entegrasyon Uzmanı olarak çalışıyorsun. Elinde doğal dille yazılan talepleri SQL sorgularına dönüştüren temel bir Gemini API betiği var. Görevin; bu betiği **1000 kayıtlık canlı bir SQLite veritabanına** bağlamak, yetkisiz veri silme/güncelleme komutlarını engelleyen bir güvenlik katmanı yazmak ve veritabanından hata alındığında Gemini'nin kendi sorgusunu otomatik düzeltmesini (Self-Correction) sağlamaktır.

TEMEL KURAL: Yapay zekânın ürettiği SQL sorgusunu doğrudan veritabanında çalıştırma. Önce güvenlik filtresinden geçir ve hata durumlarını try-except ile kontrol et.

Öğrenme Hedefleri

- Python `sqlite3` ve `random` modülleri ile bellek üzerinde 1.000 kayıtlık hacimli veri kümesi oluşturmak.
- Gemini API ile üretilen SQL sorgularını regex/string kontrolleri ile güvenlik tüzüğüne uygun süzmek.
- Veritabanı çalışma hatalarını (OperationalError) yakalayıp Gemini'ye geri besleyerek (Self-Correction) sorguyu otomatik düzelttirmek.
- LLM çıktılarını temizleyip pandas DataFrame ile yapılandırılmış tablo formatında ekrana basmak.
- Sistem çalışma performansını ve doğruluk testlerini bir `sql_asistan_raporu.txt` dosyasına kaydetmek.

120 Dakikalık Çalışma Planı

AŞAMA	SÜRE	YAPILACAKLAR
01	25 dk	SQLite veritabanını kur, döngü/random ile 1.000 adet musteriler ve siparisler verisini yukle.
02	25 dk	DROP, DELETE, UPDATE gibi tehlikeli komutları engelleyen Güvenlik Katmanını kodla.
03	35 dk	Veritabanı hata verdiğinde çalışan Otomatik Hata Düzeltme (Self-Correction Loop) mekanizmasını yaz.
04	20 dk	İnteraktif konsol döngüsünü kur ve test senaryolarını sırasıyla çalıştırıp sonuçları kontrol et.
05	15 dk	Test sonuçlarını ve gözlemleri `sql_asistan_raporu.txt` dosyasına kaydet, teslimi hazırla.

Başlangıç Kodu (sql_asistan_lab.py)

Aşağıdaki dosyayı `sql_asistan_lab.py` olarak oluşturun. Kod içerisindeki TODO alanlarını tamamlayın.

```
import sqlite3, os, re, time, random
from google import genai
from google.genai import types
# 1. VERİTABANI OLUŞTURMA VE SHADOW SCHEMA
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()
cursor.execute("CREATE TABLE musteriler (musteri_id INT, ad_soyad TEXT, sehir TEXT)")
cursor.execute("CREATE TABLE siparisler (siparis_id INT, musteri_id INT, tutar REAL, tarih TEXT)")
# TODO 1: random ile 1.000 müşteri ve 1.000 sipariş verisi üretip executemany() ile ekleyin.
# Musteriler: id 1..1000, sehirler: ['İstanbul', 'Ankara', 'İzmir', 'Bursa', 'Antalya']
# Siparisler : id 1..1000, tutar: 100-10000 TL, tarih: '2026-01-01'
def guvenlik_kontrolu(sql_kodu: str) -> bool:
 # TODO 2: DROP, DELETE, UPDATE, INSERT, ALTER komutlarını re.search() ile engelleyin.
 # Yasaklı komut varsa '[GÜVENLİK ENGELİ]' uyarısı verip False döndürün. Sadece SELECT izinli!
 pass
def clean_sql(raw_sql: str) -> str:
 # TODO 3: Gemini yanıtındaki ```sql ... ``` markdown etiketlerini regex ile temizleyin.
 pass
def sql_asistani_calistir(kullanici_talebi: str):
 # TODO 4: Gemini API entegrasyonu ve Otomatik Hata Düzeltme (Self-Correction) döngüsü.
 # - En fazla 3 deneme hakkı olan bir while döngüsü kurun.
 # - Gemini'ye istek atın, yanıtı clean_sql() ve guvenlik_kontrolu() süzgecinden geçirin.
 # - try-except sqlite3.Error: SQLite hatası alırsanız hatayı Gemini'ye geri besleyin!
 pass
# TODO 5: while True döngüsü kurarak kullanıcıdan interaktif istek alın.
# Her sorgunun icra süresini time.perf_counter() ile ölçüp konsola ve rapora yazdırın.
```

Gemini'ye Sorulacak Örnek İstek (System Instruction)

> *"Aşağıdaki veritabanı şemasına göre kullanıcının doğal dille istediği veriyi çekecek SQLite uyumlu SQL sorgusunu yaz. Şema: Musteriler(musteri_id, ad_soyad, sehir), Siparisler(siparis_id, musteri_id, tutar, tarih). Kurallar: Sadece SELECT komutu kullan. Çıktıda markdown etiketi ve açıklama olmasın."*

Deney Planı

- Döngü kullanarak 1.000 adet müşteri ve 1.000 adet sipariş kaydını bellekteki SQLite veritabanına ekleyin.
- Gemini'den gelen SQL yanıtlarının başındaki ve sonundaki markdown ifadelerini `clean_sql()` ile temizleyin.
- Sisteme veri silmeye yönelik zararlı talepler girerek Güvenlik Katmanının çalıştığını doğrulayın.
- Kasıtlı olarak hatalı talepler yazıp Otomatik Hata Düzeltme (Self-Correction) mekanizmasının çalıştığını izleyin.

Doğrulama ve Raporlama

Kodun sadece çalışması yeterli değildir. Aşağıdaki senaryolarda 1.000 kayıtlık veri üzerinde sistemin ürettiği SQL ve güvenlik tepkilerini test edin.

Rapor Dosyasının Örnek Düzeni (sql_asistan_raporu.txt)

```
AKILLI SQL ASİSTANI PERFORMANS VE GÜVENLİK RAPORU
------------------------------------------------
Çalıştırma Tarihi : 2026-10-30
Veritabanı Hacmi : 1.000 Müşteri / 1.000 Sipariş
Toplam Test Talebi : 4
Başarılı İcra : 3
Engellenen Saldırı : 1
Hata Düzeltme Sayısı: 1 (Self-Correction başarılı)
TEST DETAYLARI:
[TEST 1] 1000 Veride İcra Süresi: 0.042 saniye | Durum: BAŞARILI
[TEST 2] Güvenlik Engeli: BAŞARILI (DELETE engellendi)
[TEST 3] Hacimli Filtre: 0.038 saniye | Durum: BAŞARILI
[TEST 4] Hata Düzeltme: 2. denemede çözüldü (Sütun adı otomatik düzeltildi)
KISA YORUM VE DEĞERLENDİRME:
1.000 kayıtlık veritabanında Gemini-2.5-Flash ile SQL üretimi ortalama 1.2 saniye sürmüştür.
Üretilen SQL'lerin SQLite üzerindeki icra süresi ortalama 40ms civarındadır.
Güvenlik filtresi DELETE talebini tespit edip veritabanına ulaşmadan engellemiştir.
```

Örnek Terminal / Konsol Çıktısı

```
$ python sql_asistan_lab.py
[BİLGİ] Veritabanı oluşturuldu. 1.000 müşteri ve 1.000 sipariş yüklendi.
--------------------------------------------------------------------------------
Soru Girin (Çıkış için 'q'): Istanbul'daki musterilerin adini ve harcamasini getir.
[GEMINI] SQL Üretiliyor...
[SQL] SELECT m.ad_soyad, SUM(s.tutar) FROM musteriler m JOIN siparisler s ON m.musteri_id = s.musteri
[GÜVENLİK] Denetim Başarılı (Sadece SELECT).
[SONUÇ] (İcra Süresi: 0.042 sn)
+-----------------+------------------+
| ad_soyad | SUM(s.tutar) |
+-----------------+------------------+
| Ahmet Yılmaz | 24500.50 TL |
| Ayşe Kaya | 18200.00 TL |
+-----------------+------------------+
Soru Girin (Çıkış için 'q'): Tum siparisleri sil.
[GEMINI] SQL Üretiliyor...
[GÜVENLİK ENGELİ] Zararlı SQL tespit edildi: 'DELETE' komutuna izin verilmiyor!
```

Gerçek test sonuçlarını yazın. API anahtarını kodunuza gömmeyin (GEMINI_API_KEY ortam değişkeni kullanın). Yapay zekânın ürettiği kodu güvenlik denetiminden geçirmeden çalıştırmayın.

Teslim Edilecek Dosyalar

sql_asistan_lab.py
1.000 kayıtlık veritabanı kuran, güvenlik katmanlı ve self-correction mekanizmalı Python kodunuz.

sql_asistan_raporu.txt
1.000 verideki çalışma süreleri, engellenen güvenlik ihlalleri ve değerlendirme raporunuz.

İsteğe bağlı: Terminal çıktısı ekran görüntüsü
Çalışan sistemin ve engellenen zararlı komutların terminal ekran görüntüsü.